trevor-khwam.tabougua@gwdg.de

Trevor Khwam Tabougua

Software management with Spack

Table of contents

Learning objectives

- Understand the challenges of software installations in HPC.
- Formulate valid spack SPECS
- Install packages on the cluster using Spack.
- Manage different software versions.

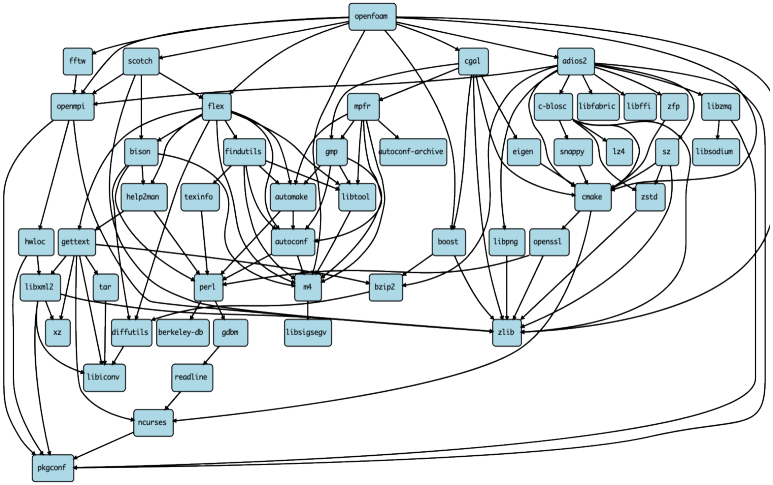
Installing software on a Cluster

- Prepackaged software typically requires administrator (root) privileges
- admins cannot install all software required by users
- software installation very complex
- different tools may need different versions
- often trade reuse and usability for performance
- Compiling on the target system often yields better performance

HPC Software is complex

- coexistence of several builds
- specific versions of compilers, MPI, libraries, dependencies
- often many dependencies

Openfoam dependency tree



HPC Software is complex

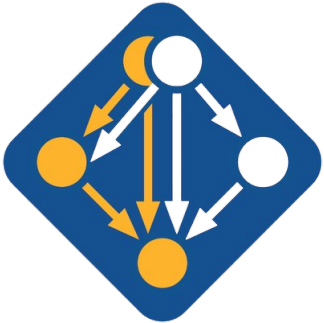
- specific versions of compilers, MPI, libraries, dependencies
- often many dependencies
- many compiling options
- users active on several clusters

Solution: **Spack**

Spack

- manages multiple builds
- takes care of dependency relationships
- drives package-level build systems
- wrapper around built systems (cmake/autotools/make etc.)
- targeted towards users, admins and developers

Spack: Supercomputer PACKage manager

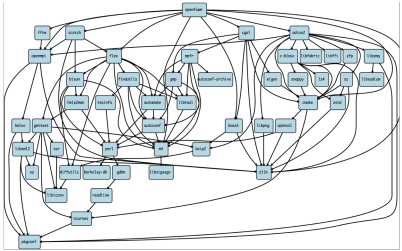


Overview

- Spack is a flexible package management system, composed primarily of Python scripts
  - 98.2% of its codebase is written in Python, according to GitHub
- Packages are maintained collaboratively by developers and the open-source community
  - As of 2024, there are over 5,953 available packages in the Spack ecosystem
  - Repository: `https://github.com/spack/spack`
  - You can still contribute to this open-source project, as the platform remains actively maintained.
    - `https://github.com/spack/spack/issues`
    - `https://github.com/spack/spack/pulls`

Spack's Build System

- Spack translates dependency graphs into unique hashes
  - Each build is assigned a unique hash identifier if all build aspects are identical
- Utilizes RPATH and PATH to manage and locate dependencies efficiently
  - Ensures all required libraries and executables are found at runtime



Using Spack on the SCC

- Most software modules on the cluster are installed using Spack
- Spack is also available as a module: `spack-user`
  - Easily search for installed software
  - Reuse existing packages to avoid redundant builds
  - Manage custom builds with your own configurations

Basic Workflow for Using Spack

- First, load the Spack module:
  - `module load spack-user`
- Enable Spack shell support by following the setup instructions provided after loading the module:
  - `source $ SPACK_USER_ROOT/share/spack/setup-env.sh`
- Install software packages using the following command:
  - `spack install <SPEC>`
- Load the installed software into your environment:
  - `spack load <SPEC>`

Understanding SPEC Syntax

- **SPECs** define the software configuration, including any constraints you specify for installation
- You can customize the installation by setting specific constraints or just use the defaults
- SPECs are flexible: only specify the options you need

`$ spack install mpileaks`	Unconstrained
`$ spack install mpileaks@3.3`	@ custom version
`$ spack install mpileaks@3.3 %gcc@4.9.3`	% custom compiler
`$ spack install mpileaks@3.3 %gcc@4.9.3 +threads`	+/-/~ build option
`$ spack install mpileaks@3.3 cppflags="-O3 -g3"`	Set compiler flags
`$ spack install mpileaks@3.3 target=cascadelake`	Set CPU architecture
`$ spack install mpileaks@3.3 ^mpich@3.2 %gcc@4.9.3`	^ dependency information

Example: Installing and Using Nano

```
gwdu102:61 23:24:18 ~ > spack install nano
[+] /opt/sw/rev/21.12/haswell/gcc-9.3.0/pkgconf-1.8.0-ktrb7r
[+] /opt/sw/rev/21.12/haswell/gcc-9.3.0/ncurses-6.2-2dvkjs
==> Installing nano-4.9-hyadazgagq6itbtp4dwnrgis3xpyrxky
==> No binary for nano-4.9-hyadazgagq6itbtp4dwnrgis3xpyrxky found: installing from source
==> Using cached archive: /usr/users/ttaboug/.spack/0.17.1/cache/_source-cache/archive/0e/0e399729d105cb1a587b4140db5cf1b23215a0886a42b215efa98137164233
a6.tar.xz
==> No patches needed for nano
==> nano: Executing phase: 'autoreconf'
==> nano: Executing phase: 'configure'
==> nano: Executing phase: 'build'
==> nano: Executing phase: 'install'
==> nano: Successfully installed nano-4.9-hyadazgagq6itbtp4dwnrgis3xpyrxky
  Fetch: 0.02s.  Build: 49.99s.  Total: 50.01s.
[+] /usr/users/ttaboug/.spack/0.17.1/install/haswell/gcc-9.3.0/nano-4.9-hyadaz
```

- To find the software and its dependencies:
  - `spack find -vdl nano`
- To install the software:
  - `spack install nano`
- To load the installed version:
  - `spack load nano`
- Verify the installation:
  - `nano –version`

The Most Important Command in Spack

`spack help`

- Provides an overview of all available Spack commands
- Useful for navigating Spack's wide array of features

`spack help <command>`

- Displays detailed information about a specific command
- Includes usage examples and available options
- A great way to get familiar with Spack's functionality

Basic Spack Commands

spack list

- Shows a list of all available packages in Spack
- Helps you discover what can be installed

spack find

- Lists all packages you have already installed
- Useful for checking what is currently available in your environment

spack compilers

- Lists all compilers Spack has detected on your system
- Important for managing builds with specific compilers

spack info

- Provides detailed information about a specific Spack package
- Includes version information, dependencies, and more

References

- Spack Documentation: `https://spack.readthedocs.io`
- Spack GitHub Repository: `https://github.com/spack/spack`
- Spack Tutorials: `https://spack-tutorial.readthedocs.io/en/latest/`